

KMIP on PKCS#11

From key server to key management service

Design · gap analysis · roadmap

41 / 53

KMIP 2.1 operations

816

tests, live against a token

74

features assessed vs market

19

gaps outside KMIP's reach

One protocol for every key store

BEFORE

N vendors, N integrations

- Every HSM and KMS exposed its own proprietary API
- Each database, backup tool and storage array needed bespoke work
- Changing vendor meant rewriting every integration

WITH KMIP

One protocol, many products

- A client speaks KMIP once and talks to any compliant server
- Oracle TDE, SQL Server, NetApp, VMware and Veeam all speak it
- Vendor substitution becomes a configuration change

What the standard actually is

- An OASIS standard: a wire protocol plus an object model, not a product
- TTLV — tag, type, length, value — a compact binary encoding carried over TLS
- Client/server, request/response, with batching; 53 operations defined in version 2.1
- Governs the whole life of a key: create, use, rotate, retire, destroy, and prove what happened

Managed objects, attributes, a lifecycle

OBJECT TYPES

What KMIP manages

- SymmetricKey
- PublicKey / PrivateKey
- Certificate
- SecretData
- OpaqueObject
- SplitKey

ATTRIBUTES

What describes them

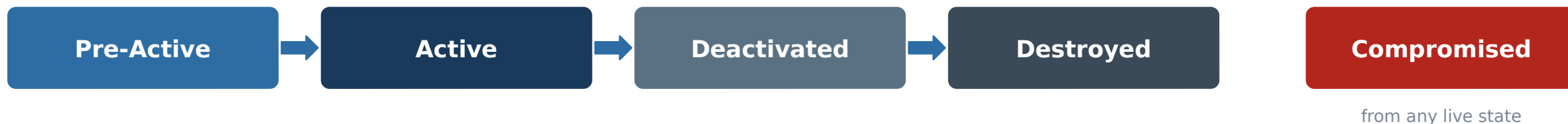
- Algorithm and length
- Cryptographic usage mask
- Name and custom x- attributes
- Links between keys
- Dates: activation, deactivation
- Owner and state

THE SUBTLE ONE

Cryptoperiod

KMIP defines no cryptoperiod attribute. The Deactivation Date is the end of the period — so enforcing a cryptoperiod means acting on a standard attribute, not inventing a parallel one.

Lifecycle



Deactivated keys may still decrypt and verify — retiring a key must not strand the data encrypted under it.

What the standard deliberately leaves out

KMIP DEFINES

Keys, and operations on keys

- Managed objects and their attributes
- The lifecycle state machine
- Cryptographic operations: encrypt, sign, MAC, derive, wrap
- Search over objects (Locate), batching, version negotiation

KMIP DOES NOT DEFINE

Everything around them

- Identities, roles, groups, grants — no user management at all
- Any HTTP or REST binding: TTLV over TCP only
- Tenancy, quotas, reporting, dashboards
- Administrative operations of any kind

The consequence

Every commercial KMS bolts an administrative plane onto its KMIP server. A KMIP layer is an interoperability surface — it is not, by itself, a product.

VERSIONS

2.1 → 3.0

2.1 (2019) is what this project implements.
3.0 CSD02 (May 2026) adds ML-KEM, ML-DSA, SLH-DSA and Encapsulate/Decapsulate.

KMIP 2.1 on a PKCS#11 token

ALONGSIDE

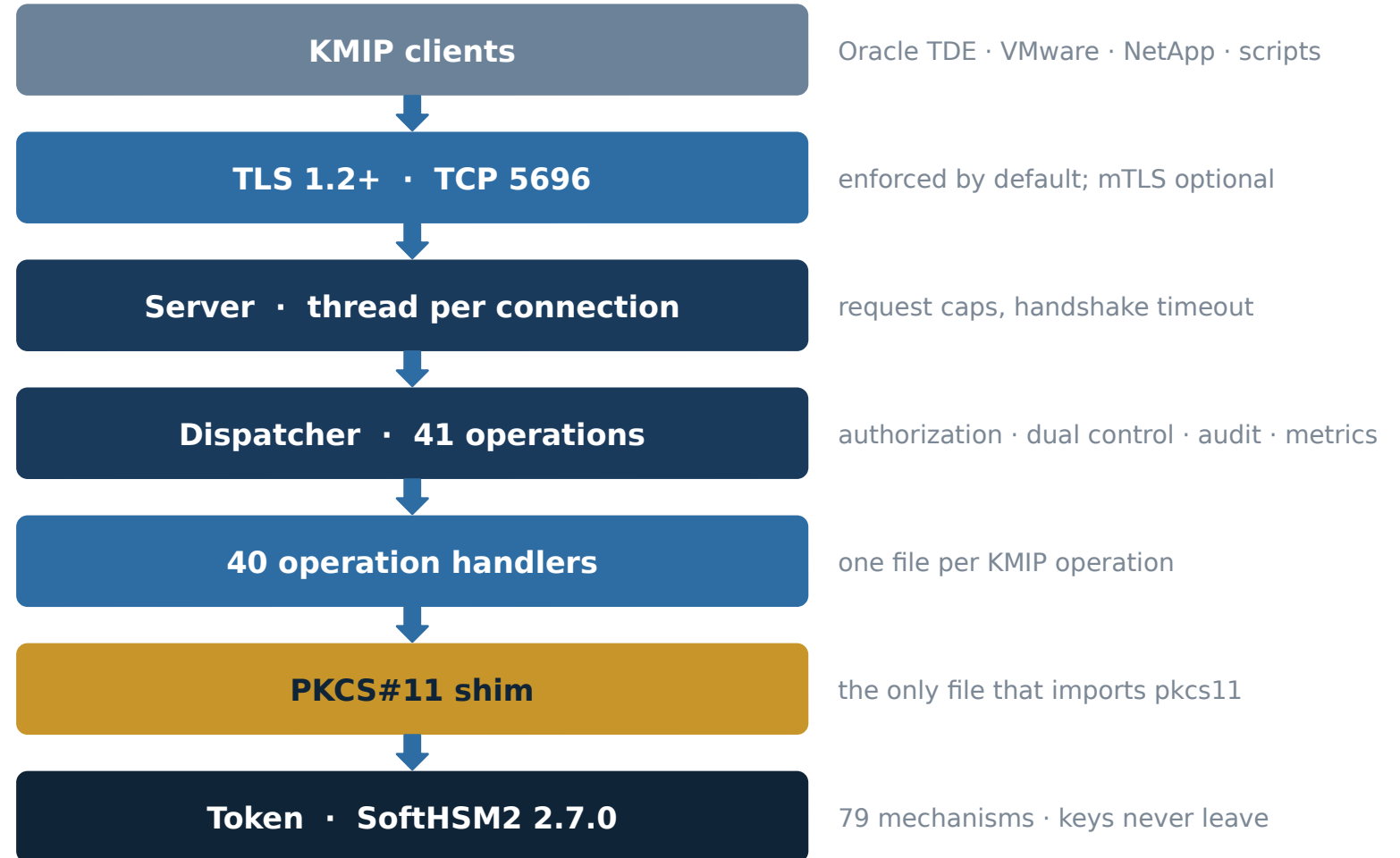
Metadata store

- SQLite, WAL mode
- KMIP attributes and state
- Identities, roles, grants
- Hash-chained audit log

SWAP POINT

Any PKCS#11 HSM

One file imports the binding. Pointing it at validated hardware needs no change above the shim — the capability probe reports whatever that token supports.



Two stores, one matched pair

IN THE TOKEN

Key material

- Generated inside the HSM, never leaves it
- Non-extractable; the usage mask is enforced by the token
- Referenced from the database only by CKA_ID

IN THE DATABASE

Everything KMIP needs that PKCS#11 has no room for

- Attributes, state, dates, names, links
- Owner, grants, groups, role allowlists, approvals
- The tamper-evident audit chain

THE EXCEPTION

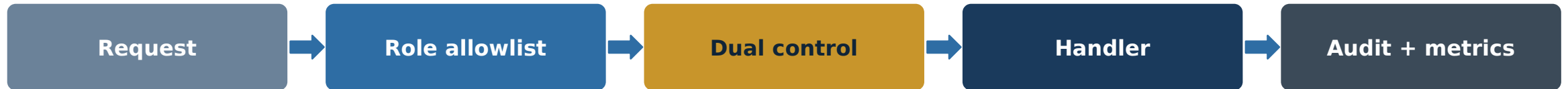
Objects with no PKCS#11 representation

SecretData, OpaqueObject and SplitKey shares have no token object behind them, so their bytes would sit in the database in the clear. They are sealed with AES-256-GCM under a master key that is generated on, and never leaves, the HSM.

Back them up together

A database restored beside a different token is not a degraded backup. It is unreadable.

Where authorization, approval and audit live



The dispatcher wraps every one of the 41 operations in this pipeline.

THE HAZARD

Bypass

Anything that calls the store directly skips all four checks. Add a second transport carelessly and dual control silently stops applying to it — which is why a shared service layer must come before a REST API.

CONCURRENCY

One session per process

A PKCS#11 session pool was built, tested, and reproducibly segfaulted: python-pkcs11 calls C_Initialize(NULL), so the library's own thread safety is never enabled. Scale with pre-fork workers, never with threads. This is permanent design, not a stopgap.

CAPABILITY PROBE

Ask the token, don't assume

The shim reads the token's real mechanism list at startup and gates every dispatch on it. An unsupported algorithm fails cleanly and by name, instead of surfacing a raw PKCS#11 error from deep inside an operation.

Why a REST layer is not optional

01

The operations do not exist

Identities, roles, grants, approvals, cryptoperiods, audit search, backups — KMIP defines none of them.

02

A browser cannot speak TTLV

Binary over raw TCP. A browser has fetch and WebSocket. Something must translate — and that translator is the API.

03

The auth models differ

KMIP re-authenticates every request from a header credential. A console needs login, session, idle timeout, CSRF, revocation.

04

The read shapes are wrong

Locate returns bare identifiers with no paging, sort or count. A console table becomes N+1 round trips.

05

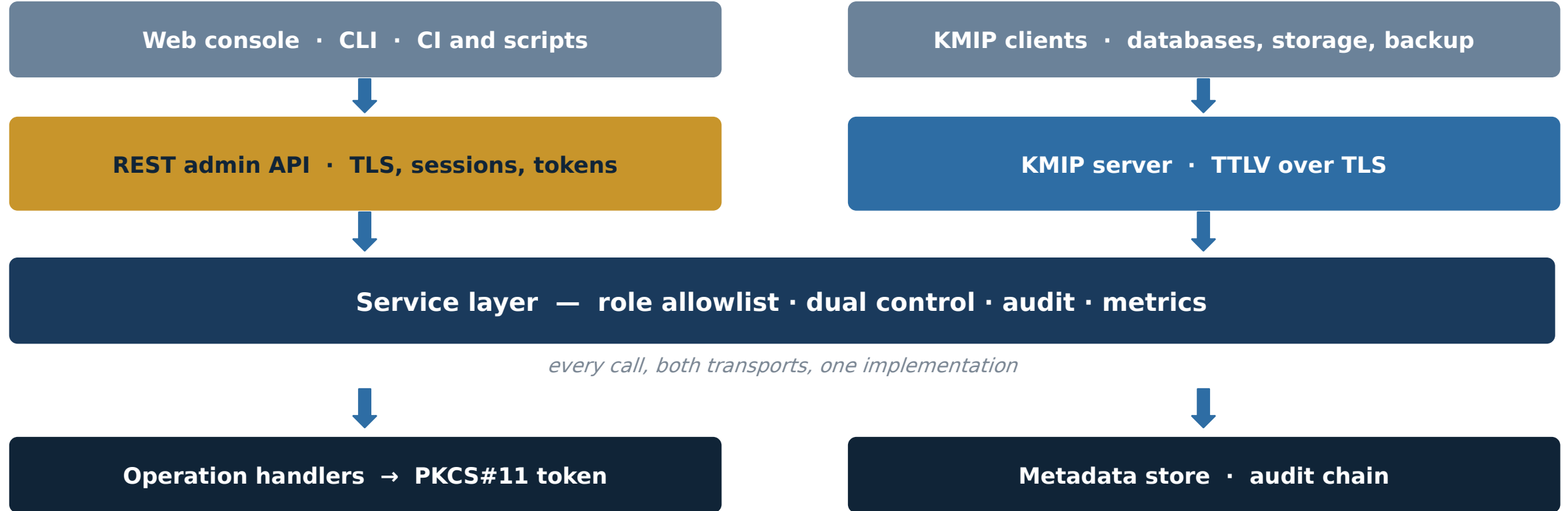
Not everything is object-centric

Approval queues, audit search, expiry reports, chain verification, health. None is an operation on a managed object.

The division of labour

KMIP stays the data plane — machines creating and using keys. REST becomes the control plane — people and CI administering the service. Duplicating crypto operations across both gives you two paths to Destroy, and two authorization implementations to keep in step.

One service layer, two transports



Endpoints: sessions and identity · keys and lifecycle · grants, groups, roles · approval queue · audit search · cryptoperiod and expiry reports · backups. No encrypt, decrypt or sign over REST in v1 — deliberately.

74 features the market expects

32

covered

16

partial

26

not covered

19

cannot be closed in KMIP

Domain	Features	Covered	Partial	Gap
Key lifecycle and cryptography	14	8	4	2
Protocol and ecosystem integration	12	1	4	7
Identity and access control	13	6	1	6
Audit, compliance and assurance	10	3	2	5
Availability, scale and recovery	10	3	3	4
Operations and observability	8	5	1	2
Platform hardening	7	6	1	0
TOTAL	74	32	16	26

What holds up

Cryptographic core

41 of 53 operations, capability-probed against the token; keys never leave the HSM.

Lifecycle and governance

State machine, cryptoperiod scheduler, scheduled rotation, dual control on Destroy and Export.

Access control

Per-identity script credentials, admin role, ownership, per-object grants, groups, per-role allowlists.

Audit

Append-only two ways — triggers and a SHA-256 chain — with retention and a verifier that names the broken row.

Data at rest

AES-256-GCM envelopes under a non-extractable HSM master key, with rotation.

Backup and recovery

Online snapshot with token pairing, and a restore that proves a stored secret decrypts.

Several of these are done properly rather than nominally: the restore proves a secret decrypts rather than assuming it, and dual control is enforced in the store, so the requester cannot self-approve however the approval arrives.

Where it falls short of a product

No REST API or console

Highest leverage: reporting, self-service, the console and most integrations are all clients of an API that does not exist.

No multi-tenancy

Names, searches, quotas and audit are global. Isolation today means one deployment per tenant.

SQLite only

No clustering, replication or DR — and the storage engine, not the protocol, is what blocks them.

No connection or rate limiting

The listener takes a backlog of 16 with no throttle. Flagged in the original review and never fixed.

Scaling stops at ~1.5×

Pre-fork workers help, but every audited operation serialises on one write lock. The price of tamper-evidence.

No SIEM export, no external anchor

Auditors ask for both first. A consistent rewrite of the local log leaves no trace.

Not FIPS or CC validated

Belongs to the token, not the code. Procurement, not engineering.

No post-quantum

Needs both a PQC token and KMIP 3.0. Neither has shipped in a release yet.

Five waves, in dependency order



How the waves are ordered

- By what each unlocks, not by visibility
- Wave 6 makes five other gaps addressable at once
- Wave 8 is a prerequisite for 9, not a parallel track
- Each wave ships independently and is useful alone

THE RULE THAT MADE PHASES 0-5 WORK

Every wave has a demonstrable gate

Not a checklist — a condition someone can watch you meet. A wave is finished when its gate is demonstrated, not when its code is written.

Wave 6 and 7 — the control plane

WAVE 6.1

Service layer extraction

Move authorization, dual control, audit and metrics out of the dispatcher into a guard both transports call.

Gate: pure refactor — all 816 tests pass unchanged, no new surface.

WAVE 6.2

Read-only REST + sessions

Login, tokens, key inventory with paging and sort, audit search, approval queue, expiry reports.

Gate: a read-only console renders real data and can change nothing.

WAVE 6.3

Write endpoints

Key lifecycle, grants, groups, cryptoperiods — all through the guard, so governance applies automatically.

Gate: a REST Destroy under dual control is refused exactly as the KMIP one is.

WAVE 6.4

Console

A single-page client of the API. No privileged back door; the console can do nothing the API forbids.

Gate: an operator completes a full day's work without the CLI.

WAVE 7.1

Directory identity + service accounts

OIDC or LDAP at the edge resolving to provisioned identities, and revocable machine credentials.

Gate: a user logs in with corporate credentials; a leaked token is revoked in one action.

WAVE 7.2

Hardening and SIEM

Connection caps and per-identity rate limiting; audit shipped as syslog or CEF.

Gate: the listener survives a flood; entries appear in the SIEM within a minute.

Wave 8 to 10 — and what stays out

8

Scale and availability

A PostgreSQL backend is the prerequisite: SQLite has no multi-writer story, so clustering, replication and DR all wait behind it. HSM failover follows, once a second token exists to test against.

9

Multi-tenancy

A tenant boundary reaches into every query, the audit log and the CLI — a wave of its own, not a flag. Worth building when there is a second tenant to serve; until then one deployment per tenant is a real answer.

10

Compliance and post-quantum

Swap in a validated token and run the suite against it. Adopt KMIP 3.0 and the PQC algorithms once a token ships them in a release — vendor-range enums would work locally and interoperate with nothing.

Deliberately out of scope

- Tokenization and FPE — a separate product, not a KMS feature.
- Being a certificate authority — front it with a real CA instead.
- A PKCS#11 provider for applications — only if demand appears.

Nothing ships that cannot be demonstrated. Where something could not be tested here, the roadmap says so rather than counting it as done.